

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
Dipartimento di Ingegneria e Scienze Informatiche

Cluster Kubernetes k3s su Raspberry Pi

High Availability: 3 Master + 3 Worker



Foto del cluster fisico: 6 Raspberry Pi 400 nel rack

Corso di **Virtualizzazione ed Integrazione di Sistemi**

Indice

Indice	2
1 Panoramica ed inventario del Cluster Raspberry	4
1.1 Inventario hardware e ruoli	4
2 Preparazione del Sistema Operativo	7
2.1 Flashing e configurazione iniziale	7
2.2 Aggiornamento sistema e dipendenze	7
2.3 Abilitazione cgroup v2 e memory cgroup	7
2.4 Configurazione <code>/etc/hosts</code>	8
2.5 Installazione Docker	8
3 kube-vip - Virtual IP per Alta Disponibilità (HA)	9
3.1 Installazione kube-vip come static pod	9
4 Installazione del Cluster k3s	11
4.1 Primo Master Node (<code>rpimaster00</code>)	11
4.1.1 Recupero del token e kubeconfig	11
4.2 Secondo e Terzo Master Node	12
4.3 Installazione Worker Nodes	13
4.4 Verifica completa del cluster	14
5 Workloads: Nginx Ingress, App Demo, HPA	15
5.1 Nginx Ingress Controller	15
5.2 Applicazione Demo testing - Web + API + Redis	16
5.3 HPA - Horizontal Pod Autoscaling	18
6 Monitoring con Prometheus e Grafana	21
6.1 Installazione kube-prometheus-stack via Helm	21
6.2 Dashboard Grafana	22
7 Load Testing e Test di Failover	24
7.1 Load Testing con k6	24
7.2 Test di Failover - High Availability	25
7.2.1 Fallimento di un Worker	25
7.3 Scenario: 1 Master Cade	26
7.4 Scenario: 2 Master in failover	27
8 Simulazione Split-Brain via Network Partition	29
8.1 Topologia del cluster prima del test	29
8.2 Preparazione del monitoraggio	29
8.3 Fase 1 - Fotografia dello stato iniziale	30
8.4 Fase 2 - Avvio del monitoring continuo	31

8.5	Fase 3 - creazione della partizione di rete	31
8.6	Fase 4 - Osservazione	32
8.6.1	Timeline degli eventi osservati	32
8.7	Fase 5 - interrogazione durante la partizione	33
8.8	Fase 6 - Rimozione della partizione	33
8.9	Fase 7 - Osservazione della ricongiunzione	34
8.10	Recupero da situazioni anomale	35
9	Conclusioni	36
A	Riferimenti e Risorse	37

1. Panoramica ed inventario del Cluster Raspberry

Il progetto consiste nella realizzazione di un cluster Kubernetes **k3s** ad alta disponibilità (HA) su sei Raspberry Pi 400, con distribuzione 3 master + 3 worker. La scelta di k3s rispetto a k8s classico è motivata dall'ottimizzazione per hardware ARM e dalla sua leggerezza (singolo binario da circa 70MB), che lo rende ideale per ambienti embedded come nel nostro caso.

1.1 Inventario hardware e ruoli

Tabella 1.1: Inventario dell'infrastruttura del cluster

Hostname	Ruolo	IP statico	RAM	Arch.
rpimaster00	Control Plane #1	10.39.100.6	4 GB	ARM64
rpimaster01	Control Plane #2	10.39.100.3	4 GB	ARM64
rpimaster02	Control Plane #3	10.39.100.4	4 GB	ARM64
rpinode00	Worker	10.39.100.2	4 GB	ARM64
rpinode01	Worker	10.39.100.7	4 GB	ARM64
rpinode02	Worker	10.39.100.12	4 GB	ARM64
HP2626	Switch Fast Ethernet	—	—	24 porte
k3s-vip	Virtual IP (kube-vip)	10.39.100.250	—	—
gateway	Default gateway	10.39.100.1	—	—

Lo switch utilizzato è un HP ProCurve 2626 (J4900B), dotato di 24 porte Fast Ethernet 10/100 Mbps e impiegato per l'interconnessione dei nodi Raspberry Pi.

Control Plane (Master) vs Worker in Kubernetes

In Kubernetes il **Control Plane** (master) ospita i componenti che gestiscono lo stato del cluster:

- *API Server*: unico punto di accesso alle API Kubernetes.
- *etcd*: database distribuito chiave-valore che memorizza l'intero stato del cluster.
- *Scheduler*: decide su quale nodo eseguire i nuovi pod.
- *Controller Manager*: garantisce che lo stato reale del cluster corrisponda a quello desiderato.

I **Worker node** eseguono invece i carichi applicativi tramite:

- *kubelet*: agente che comunica con il Control Plane e gestisce i container localmente tramite *containerd*.
- *kube-proxy*: configura le regole di rete (*iptables/IPVS*) necessarie al funzionamento dei *Service*.

La distribuzione **k3s** integra tutti questi componenti.

Alta Disponibilità (High Availability) e Raft Consensus

Con un singolo master, se questo cade, il cluster diventa non gestibile, anche se in realtà i pod esistenti su ogni node worker continuano ma non si possono schedulare nuovi né fare rollout.

Con **3 master in modalità di High Availability**, tutti e tre eseguono i componenti del control plane ed etcd.

etcd usa il protocollo **Raft** per eleggere un leader e replicare ogni scrittura: il cluster è operativo finché esiste un quorum (ovvero $> 50\%$ dei master disponibili). Con 3 master, il quorum è $2/3$, dunque, nel nostro caso, possiamo tollerare la perdita di 1 master senza alcuna interruzione del cluster.

Per perdere il quorum servono 2 master giù contemporaneamente: solo a quel punto etcd entrerà in modalità **read-only**.

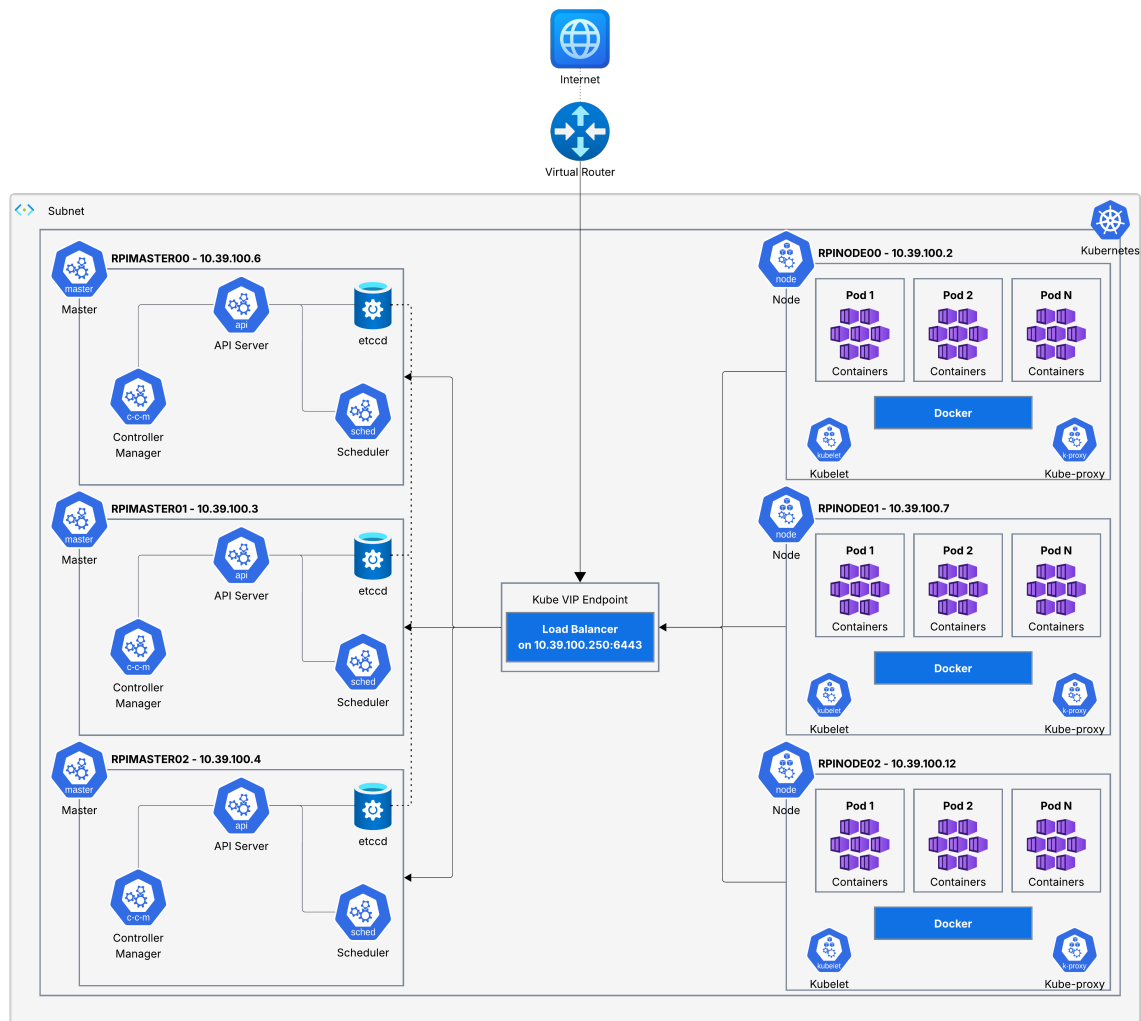


Figura 1.1: Schema architetturale del cluster: master, worker, VIP kube-vip.

2. Preparazione del Sistema Operativo

Su tutti e sei i Raspberry Pi viene installato **Raspberry Pi OS Lite 64-bit**, in versione headless per minimizzare il consumo di risorse.

Usare la versione **64-bit** in quanto k3s su ARM64 è più stabile e performante rispetto ad ARM32 e comporta meno problemi di gestione della diversificazione delle architetture.

2.1 Flashing e configurazione iniziale

La SD card di ogni Pi 400 viene preparata con *Raspberry Pi Imager*, configurando SSH, hostname e utente.

2.2 Aggiornamento sistema e dipendenze

Da eseguire su **tutti** i nodi al primo avvio:

Listing 2.1: Tutti i nodi

```
# aggiornamento di tutti i pacchetti e del kernel
sudo apt update && sudo apt full-upgrade -y

# installiamo alcune utilities necessarie
sudo apt install -y curl wget git nfs-common
```

2.3 Abilitazione cgroup v2 e memory cgroup

Kubernetes richiede i *cgroup* per isolare le risorse (come CPU, RAM, etc..) di ogni container. Su Raspberry Pi OS devono essere abilitati esplicitamente nel bootloader:

Listing 2.2: Tutti i nodi

```
# aggiungiamo i parametri al cmdline del kernel
sudo sed -i '$ s/$/ cgroup_enable=cpuset cgroup_memory=1 \
cgroup_enable=memory/' /boot/firmware/cmdline.txt

# verifichiamo che la riga sia stata modificata correttamente
cat /boot/firmware/cmdline.txt
```

cgroup (Control Groups)

I cgroup sono una funzionalità del kernel Linux che limita, isola e tiene traccia dell'utilizzo delle risorse per gruppi di processi. Kubernetes usa i cgroup per implementare i *resource requests/limits* dei pod.

Ad esempio quando andremo a specificare **resources.limits.memory: 256Mi**, è il cgroup che forza quel limite a livello kernel. Senza cgroup memory abilitato, il kubelet si rifiuta di avviarsi

o funziona in modo anomalo.

2.4 Configurazione /etc/hosts

La risoluzione dei nomi tra i Pi avviene tramite il file `/etc/hosts` su ogni nodo, evitando la dipendenza da un DNS esterno:

Listing 2.3: Tutti i nodi

```
sudo tee /etc/hosts > /dev/null <<'EOF'
# Cluster RPI 400
10.39.100.6      rpimaster00
10.39.100.3      rpimaster01
10.39.100.4      rpimaster02
10.39.100.2      rpinode00
10.39.100.7      rpinode01
10.39.100.12     rpinode02
10.39.100.250    k3s-vip
EOF
```

2.5 Installazione Docker

Docker viene installato per generare il manifest di kube-vip:

Listing 2.4: Tutti i master

```
curl -fsSL https://get.docker.com | sh

# aggiunta dell'utente al gruppo docker
sudo usermod -aG docker $USER
newgrp docker

# verifichiamo la versione di Docker installata
docker - version
```


3. kube-vip - Virtual IP per Alta Disponibilità (HA)

Kube-vip e il Virtual IP

Con 3 master, i worker e i client kubectl devono sapere a quale API Server connettersi. Se puntano all'IP fisico di `rpimaster00` (10.39.100.3) e questo va in failover, tutto smette di funzionare.

kube-vip crea un *Virtual IP* (VIP) - nel nostro caso 10.39.100.250 - che flotta tra i master: è sempre attivo sul master “leader” corrente.

Se il master leader cade, kube-vip trasferisce automaticamente il VIP a uno degli altri master tramite il protocollo **ARP**. In questo modo i worker si connettono sempre allo stesso IP, indipendentemente da quale master sia attivo in quel momento.

3.1 Installazione kube-vip come static pod

kube-vip viene eseguito come **static pod**, un pod gestito direttamente dal kubelet locale, senza passare per l'API Server. È il modo in cui k3s garantisce che il VIP sia disponibile anche durante il bootstrap ed istanziazione del cluster:

Listing 3.1: Tutti i master

```
# creiamo la directory per i manifest statici
sudo mkdir -p /var/lib/rancher/k3s/agent/pod-manifests/

# definizione delle variabili
VIP=10.39.100.250
INTERFACE=eth0
KVVERSION=$(curl -sL \
  https://api.github.com/repos/kube-vip/kube-vip/releases \
  | grep tag_name | head -1 | cut -d'"' -f4)

echo "Versione kube-vip: $KVVERSION"

# generiamo il manifest tramite container kube-vip
sudo docker run -network host -rm \
  ghcr.io/kube-vip/kube-vip:$KVVERSION \
  manifest pod \
  - interface $INTERFACE \
  - address $VIP \
  - controlplane \
  - arp \
  - leaderElection \
  | sudo tee \
  /var/lib/rancher/k3s/agent/pod-manifests/kube-vip.yaml
```

ARP mode vs BGP mode

kube-vip supporta due modalità di failover.

ARP mode (nostro caso): quando il master leader acquisisce il VIP, invia un *gratuitous ARP* sulla rete LAN (passando per lo switch) per annunciare che il VIP appartiene al suo MAC address. Lo switch aggiorna la tabella ARP e tutto il traffico diretto al VIP arriva al nuovo leader.

BGP mode è più sofisticata e viene usata in ambienti cloud/datacenter con router BGP.

4. Installazione del Cluster k3s

4.1 Primo Master Node (rpimaster00)

Come funziona l'installazione k3s HA

Il primo master viene installato con il flag `-cluster-init`, in quanto esso dice a k3s di inizializzare un nuovo **cluster etcd** da zero.

I master successivi vengono installati con `-server` puntando al VIP, e usano un *token segreto* condiviso per autenticarsi e unirsi al **cluster etcd esistente**.

I parametri TLS-SAN (Subject Alternative Names) sono fondamentali siccome essi permettono connessioni tramite VIP, hostname e IP diversi verso lo stesso API Server.

Listing 4.1: Solo su rpimaster00

```
# generiamo un token segreto condiviso (possiamo anche
# specificarne uno nostro non randomico)
K3S_TOKEN=$(openssl rand -hex 32)

# installiamo k3s come primo master con etcd embedded
curl -sfL https://get.k3s.io | \
  K3S_TOKEN=$K3S_TOKEN \
  INSTALL_K3S_EXEC="server \
    - cluster-init \
    - tls-san 10.39.100.250 \
    - tls-san k3s-vip \
    - tls-san 10.39.100.6 \
    - tls-san rpimaster00 \
    - disable servicelb \
    - disable traefik \
    - flannel-backend=vxlan \
    - node-ip=10.39.100.6 \
    - cluster-cidr=10.42.0.0/16 \
    - service-cidr=10.43.0.0/16 \
    - write-kubeconfig-mode 644" \
  sh -

# attendiamo che il servizio k3s sia attivo
sudo systemctl status k3s
```

4.1.1 Recupero del token e kubeconfig

Listing 4.2: Solo su rpimaster00

```
# il token precedentemente creato viene salvato e sara'
# necessario per far joinare altri nodi
sudo cat /var/lib/rancher/k3s/server/node-token
```

```
# Kubeconfig (salva per usare kubectl dal PC host)
sudo cat /etc/rancher/k3s/k3s.yaml

# verifichiamo che il rpimaster00 sia in stato di 'Ready'
sudo kubectl get nodes

# verifichiamo i componenti del control plane
sudo kubectl get pods -n kube-system
```

4.2 Secondo e Terzo Master Node

I master successivi si uniscono al cluster puntando al **VIP kube-vip** e, come prima descritto, non direttamente a `rpimaster00`. Questo garantisce che se `rpimaster00` andasse in failover in futuro, i master rimanenti siano già connessi al cluster tramite VIP:

Listing 4.3: su `rpimaster01 - 10.39.100.3`

```
K3S_TOKEN=<token-copiato-da-rpimaster00>

curl -sfL https://get.k3s.io | \
  K3S_TOKEN=$K3S_TOKEN \
  INSTALL_K3S_EXEC="server \
    - server https://10.39.100.250:6443 \
    - tls-san 10.39.100.250 \
    - tls-san k3s-vip \
    - tls-san 10.39.100.3 \
    - tls-san rpimaster01 \
    - disable servicelb \
    - disable traefik \
    - flannel-backend=vxlan \
    - node-ip=10.39.100.3" \
  sh -
```

Listing 4.4: su `rpimaster02 - 10.39.100.4`

```
K3S_TOKEN=<token-copiato-da-rpimaster00>

curl -sfL https://get.k3s.io | \
  K3S_TOKEN=$K3S_TOKEN \
  INSTALL_K3S_EXEC="server \
    - server https://10.39.100.250:6443 \
    - tls-san 10.39.100.250 \
    - tls-san k3s-vip \
    - tls-san 10.39.100.4 \
    - tls-san rpimaster02 \
    - disable servicelb \
    - disable traefik \
    - flannel-backend=vxlan \
    - node-ip=10.39.100.4" \
  sh -

# una volta joinati, devono comparire 3 nodi control-plane
```

```

sudo kubectl get nodes

# controlliamo lo stato di tutti e 3 i membri etcd
sudo ETCDCTL_API=3 etcdctl \
  - endpoints=https://127.0.0.1:2379 \
  - cacert=/var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  - cert=/var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  - key=/var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  endpoint status - cluster -w table

```

etcd Raft Consensus con 3 master

Con 3 nodi etcd, il protocollo **Raft** elegge un leader.

Ogni scrittura deve essere confermata dalla maggioranza dei nodi (quorum = 2/3). Dunque, ad esempio, se **1 master** cadesse, il cluster etcd rimarrebbe completamente operativo in **lettura** e **scrittura**.

Nel caso in cui 2 master andassero in failover contemporaneamente, etcd perderebbe il quorum e diventerebbe *read-only*.

Tre master è il minimo raccomandato per vera fault tolerance anche in scrittura.

4.3 Installazione Worker Nodes

Il ruolo del Worker

Su ogni worker gira **kubelet**, un agente che riceve istruzioni dall'API Server e gestisce il ciclo di vita dei container tramite **containerd**.

Invece, **kube-proxy** imposta le regole iptables per il routing dei Service.

Per ultimo, il **Flannel agent** gestisce il networking overlay VXLAN tra pod su nodi diversi. Se un worker andasse in failover, solo i pod su quel nodo verrebbero persi.

Listing 4.5: Su ogni worker (adattare NODE_IP e NODE_NAME)

```

NODE_IP=10.39.100.2    # cambiarlo per ogni nodo
NODE_NAME=rpinode00    # cambiarlo per ogni nodo
K3S_TOKEN=<token-del-cluster> # stesso di prima

curl -sfl https://get.k3s.io | \
  K3S_TOKEN=$K3S_TOKEN \
  K3S_URL=https://10.39.100.250:6443 \
  INSTALL_K3S_EXEC="agent \
    - node-ip=$NODE_IP \
    - node-name=$NODE_NAME" \
  sh -

# verifichiamo che il servizio sia avviato correttamente
sudo systemctl status k3s-agent

```

4.4 Verifica completa del cluster

Listing 4.6: Da rpimaster00

```
# listing di tutti i nodi con ruolo e versione
kubectl get nodes -o wide

# verifica tutti i pod di sistema
kubectl get pods -A

# Health check dell'API Server
kubectl get -raw='/healthz '

# Verifica accesso da PC locale
scp rpimaster00@rpimaster00.local:/etc/rancher/k3s/k3s.yaml
  ~/.kube/config
sed -i 's/127.0.0.1/10.39.100.250/g' ~/.kube/config
kubectl get nodes
```

L'output atteso con tutti i nodi pronti:

Listing 4.7: Output atteso - kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
rpimaster00	Ready	control-plane,etcd	...	v1.x.x
rpimaster01	Ready	control-plane,etcd	...	v1.x.x
rpimaster02	Ready	control-plane,etcd	...	v1.x.x
rpinode00	Ready	<none>	...	v1.x.x
rpinode01	Ready	<none>	...	v1.x.x
rpinode02	Ready	<none>	...	v1.x.x

5. Workloads: Nginx Ingress, App Demo, HPA

5.1 Nginx Ingress Controller

Ingress Controller

Un *Service* Kubernetes di tipo *NodePort* espone un'applicazione su una porta (esempio 30000 - 32767) di ogni worker. Con un **Ingress Controller** (Nginx nel nostro caso), un unico IP/porta (80/443) può servire centinaia di applicazioni diverse in base all'hostname o al path URL.

L'Ingress Controller legge le risorse *Ingress* e configura dinamicamente Nginx come reverse proxy.

Listing 5.1: da rpimaster00 - Installazione via Helm

```
# installiamo Helm - package manager per Kubernetes
curl https://raw.githubusercontent.com/helm/helm/main/scripts/
  get-helm-3 | bash

# aggiungiamo repo ingress-nginx
helm repo add ingress-nginx \
  https://kubernetes.github.io/ingress-nginx
helm repo update

# installiamo ingress-nginx in modalita' DaemonSet
helm install ingress-nginx ingress-nginx/ingress-nginx \
  - namespace ingress-nginx \
  - create-namespace \
  - set controller.kind=DaemonSet \
  - set controller.hostNetwork=true \
  - set controller.hostPort.enabled=true \
  - set controller.service.type=ClusterIP \
  - set controller.nodeSelector."kubernetes\.io/os"=linux

# verifichiamo che i pod siano in 'Running' su ogni node
kubectl get pods -n ingress-nginx -o wide
```

5.2 Applicazione Demo testing - Web + API + Redis

Possiamo eseguire il deploy di una app con **frontend Nginx**, **backend API Node.js** e **Redis**. Questo stack ci permette di osservare il load balancing tra le repliche e la comunicazione inter-pod.

Listing 5.2: demo-app.yaml

```
- -
# Custom Namespace
apiVersion: v1
kind: Namespace
metadata:
  name: demo

- -
# Redis
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: demo
spec:
  serviceName: redis
  replicas: 1
  template:
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        resources:
          requests: {memory: "64Mi", cpu: "50m"}
          limits:   {memory: "128Mi", cpu: "200m"}

- -
# Ingress routing HTTP
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
  namespace: demo
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
  - host: demo.cluster.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: {name: frontend, port: {number: 80}}
      - path: /api(/|$)(.*)
```



```
pathType: ImplementationSpecific
backend:
  service: {name: api, port: {number: 80}}
```

Listing 5.3: Deploy e verifica

```
kubectl apply -f demo-app.yaml

# osserviamo il rollout in real-time
kubectl rollout status deployment/api -n demo
kubectl rollout status deployment/frontend -n demo

# guardiamo su quali nodi sono distribuiti i pod creati
kubectl get pods -n demo -o wide

# test al volo via HTTP via curl
curl -H "Host: demo.cluster.local" http://10.39.100.250/
curl -H "Host: demo.cluster.local" http://10.39.100.250/api/
```

```
rpimaster00@rpimaster00:~$ kubectl get pods -n ingress-nginx -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
ingress-nginx-controller-4kjlz      0/1     ContainerCreating  0      47s   10.39.100.7     rpimaster01      <none>            <none>
ingress-nginx-controller-9hngl      0/1     ContainerCreating  0      47s   10.39.100.3     rpimaster01      <none>            <none>
ingress-nginx-controller-9k487      0/1     ContainerCreating  0      47s   10.39.100.1     rpimaster02      <none>            <none>
ingress-nginx-controller-jc9rs      0/1     ContainerCreating  0      47s   10.39.100.2     rpimaster00      <none>            <none>
ingress-nginx-controller-mc6sc      0/1     ContainerCreating  0      47s   10.39.100.6     rpimaster00      <none>            <none>
ingress-nginx-controller-zwkgs      0/1     ContainerCreating  0      47s   10.39.100.4     rpimaster02      <none>            <none>
rpimaster00@rpimaster00:~$ kubectl get pods -n ingress-nginx -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
ingress-nginx-controller-4kjlz      1/1     Running    0          82s   10.39.100.7     rpimaster01      <none>            <none>
ingress-nginx-controller-9hngl      1/1     Running    0          82s   10.39.100.3     rpimaster01      <none>            <none>
ingress-nginx-controller-9k487      1/1     Running    0          82s   10.39.100.1     rpimaster02      <none>            <none>
ingress-nginx-controller-jc9rs      1/1     Running    0          82s   10.39.100.2     rpimaster00      <none>            <none>
ingress-nginx-controller-mc6sc      1/1     Running    0          82s   10.39.100.6     rpimaster00      <none>            <none>
ingress-nginx-controller-zwkgs      1/1     Running    0          82s   10.39.100.4     rpimaster02      <none>            <none>
rpimaster00@rpimaster00:~$ nano demo-app.yaml
rpimaster00@rpimaster00:~$ kubectl apply -f demo-app.yaml
namespace/demo created
statefulset.apps/redis created
service/redis created
deployment.apps/api created
service/api created
deployment.apps/frontend created
configmap/frontend-html created
service/frontend created
ingress.networking.k8s.io/demo-ingress created
rpimaster00@rpimaster00:~$ kubectl rollout status deployment/api -n demo
Waiting for deployment "api" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "api" rollout to finish: 2 of 3 updated replicas are available...
deployment "api" successfully rolled out
rpimaster00@rpimaster00:~$
```

Figura 5.1: Deployment pods ingress-nginx

```
rpimaster00@rpimaster00:~$ kubectl rollout status deployment/api -n demo
Waiting for deployment "api" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "api" rollout to finish: 2 of 3 updated replicas are available...
deployment "api" successfully rolled out
rpimaster00@rpimaster00:~$ kubectl rollout status deployment/frontend -n demo
deployment "frontend" successfully rolled out
rpimaster00@rpimaster00:~$ kubectl get pods -n demo -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
api-5c67599cb4-g7s58                1/1     Running    0          4m31s  10.42.4.4       rpimaster01      <none>            <none>
api-5c67599cb4-pwvjw                1/1     Running    0          4m31s  10.42.5.61      rpimaster02      <none>            <none>
api-5c67599cb4-zwbj5                1/1     Running    0          4m31s  10.42.3.14      rpimaster00      <none>            <none>
frontend-bb97f5658-dzdfz            1/1     Running    0          4m31s  10.42.5.62      rpimaster02      <none>            <none>
frontend-bb97f5658-wgt72            1/1     Running    0          4m31s  10.42.4.3       rpimaster01      <none>            <none>
frontend-bb97f5658-z7lvb            1/1     Running    0          4m31s  10.42.2.32      rpimaster02      <none>            <none>
frontend-bb97f5658-zqntx            1/1     Running    0          4m31s  10.42.3.15      rpimaster00      <none>            <none>
redis-0                              1/1     Running    0          4m32s  10.42.3.17      rpimaster00      <none>            <none>
```

Figura 5.2: Esempio di Rollout deployment

```

prometheus-prometheus-stack-kube-prom-prometheus-0      1/2      Running    0          57s
prometheus-prometheus-stack-kube-prom-prometheus-0      1/2      Running    0          72s
prometheus-prometheus-stack-kube-prom-prometheus-0      2/2      Running    0          73s
prometheus-stack-grafana-6c6f86557d-gfttv               2/3      Running    0          92s
^Crpimaster00@rpimaster00:~$ kubectl get pods -n monitoring
NAME                                READY    STATUS    RESTARTS   AGE
alertmanager-prometheus-stack-kube-prom-alertmanager-0  2/2      Running   0          101s
prometheus-prometheus-stack-kube-prom-prometheus-0      2/2      Running   0          100s
prometheus-stack-grafana-6c6f86557d-gfttv               2/3      Running   0          112s
prometheus-stack-kube-prom-operator-78c5c76f74-6h5kz    1/1      Running   0          113s
prometheus-stack-kube-state-metrics-7b9d949668-x7492    1/1      Running   0          112s
prometheus-stack-prometheus-node-exporter-2tln9         1/1      Running   0          112s
prometheus-stack-prometheus-node-exporter-76m49         1/1      Running   0          113s
prometheus-stack-prometheus-node-exporter-cczzv         1/1      Running   0          112s
prometheus-stack-prometheus-node-exporter-cfl9w         1/1      Running   0          113s
prometheus-stack-prometheus-node-exporter-w65g5         1/1      Running   0          112s
prometheus-stack-prometheus-node-exporter-wxhwX         1/1      Running   0          113s
rpimaster00@rpimaster00:~$ kubectl get pods -n monitoring --watch
NAME                                READY    STATUS    RESTARTS   AGE
alertmanager-prometheus-stack-kube-prom-alertmanager-0  2/2      Running   0          106s
prometheus-prometheus-stack-kube-prom-prometheus-0      2/2      Running   0          105s
prometheus-stack-grafana-6c6f86557d-gfttv               2/3      Running   0          117s
prometheus-stack-kube-prom-operator-78c5c76f74-6h5kz    1/1      Running   0          118s
prometheus-stack-kube-state-metrics-7b9d949668-x7492    1/1      Running   0          117s
prometheus-stack-prometheus-node-exporter-2tln9         1/1      Running   0          117s
prometheus-stack-prometheus-node-exporter-76m49         1/1      Running   0          118s
prometheus-stack-prometheus-node-exporter-cczzv         1/1      Running   0          117s
prometheus-stack-prometheus-node-exporter-cfl9w         1/1      Running   0          118s
prometheus-stack-prometheus-node-exporter-w65g5         1/1      Running   0          117s
prometheus-stack-prometheus-node-exporter-wxhwX         1/1      Running   0          118s
prometheus-stack-grafana-6c6f86557d-gfttv               3/3      Running   0          2n3s

```

Figura 5.3: Monitoring dei pods deployati

```

rpimaster00@rpimaster00:~$ curl -H "Host: demo.cluster.local" http://10.39.100.250/
<!DOCTYPE html>
<html><head>
<title>k3s Cluster Demo</title>
<style>
body{font-family:monospace;background:#0f1117;color:#e2e8f0;
display:flex;align-items:center;justify-content:center;
min-height:100vh;margin:0;}
.card{background:#161b27;border:1px solid #2a3248;border-radius:12px;
padding:2rem;text-align:center;max-width:400px;}
h1{color:#a78bfa;margin-bottom:1rem;}
.badge{background:#4c1d95;color:#a78bfa;padding:4px 12px;
border-radius:20px;font-size:12px;}
</style>
</head><body>
<div class="card">
<h1>* k3s Cluster</h1>
<span class="badge">12 Raspberry Pi</span>
<p style="margin-top:1rem;color:#94a3b8">Frontend attivo e servito da Nginx</p>
<p><a href="/api" style="color:#60a5fa">Prova l'API</a></p>
</div>
</body></html>
rpimaster00@rpimaster00:~$ curl -H "Host: demo.cluster.local" http://10.39.100.250/api/
{
  "path": "/",

```

Figura 5.4: Test delle API tramite curl eseguito sul nodo rpimaster00.

5.3 HPA - Horizontal Pod Autoscaling

Horizontal Pod Autoscaler

L'HPA è un controller Kubernetes che monitora le metriche di un Deployment (CPU, RAM, o metriche custom) e scala automaticamente il numero di repliche. Quando il consumo medio di CPU supera il target (es. 60%), l'HPA aumenta le repliche per distribuire il carico. Funziona tramite il *Metrics Server* che raccoglie dati da kubelet dei nodi ogni circa 60 secondi.

Listing 5.4: su rpimaster00

```

# installiamo il Metrics Server
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/\
releases/latest/download/components.yaml

# Su Raspberry Pi serve il flag - kubelet-insecure-tls
kubectl patch deployment metrics-server -n kube-system \
  - type=json \
  -p='[{"op": "add",

```

```

        "path":"/spec/template/spec/containers/0/args/-",
        "value":" - kubelet-insecure-tls"}] '

# verifichiamo le metriche disponibili
kubectl top nodes
kubectl top pods -n demo

# istanziamo un HPA per il backend API
kubectl autoscale deployment api -n demo \
  - cpu-percent=60 \
  - min=3 \
  - max=15

# osserviamo lo stato dell'HPA
kubectl get hpa -n demo
kubectl describe hpa api -n demo

```

Video degli esperimenti Per una dimostrazione pratica del comportamento dell'Horizontal Pod Autoscaler e della distribuzione dei pod nel cluster, sono disponibili i seguenti video:

- k6-hpa
- cluster-pod-capacity-foreach-node
- k6-hpa-slowng-down

```

pinaster00@pinaster00:~$ kubectl autoscale deployment api -n demo \
--cpu=60 \
--min=3 \
--max=15
error: failed to create autoscaling/v2 HPA and the configuration is incompatible with autoscaling/v1: %!w(<nil>)
pinaster00@pinaster00:~$ kubectl autoscale deployment api -n demo --cpu-percent=60 --min=3 --max=15
flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity format (e.g., '70%' for utilization or '500m' for millicPU).
error: failed to create autoscaling/v2 HPA and the configuration is incompatible with autoscaling/v1: empty input
pinaster00@pinaster00:~$ kubectl get hpa -n demo
NAME REFERENCE TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
api   Deployment/api  cpu: 6%/60%  3        15       3         91s
pinaster00@pinaster00:~$ kubectl describe hpa api -n demo
Name:
Namespace:
Labels:
Annotations:
CreationTimestamp:
Reference:
Metrics:
  resource cpu on pods (as a percentage of request): 6% (3m) / 60%
Min replicas:
Max replicas:
Deployment pods:
Conditions:
  Type            Status  Reason
  ----            -
AbleToScale      True    ScaleDownStabilized recent recommendations were higher than current one, applying the highest recent recommendation
ScalingActive     True    ValidMetricFound    the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of request)
ScalingLimited    False   DesiredWithinRange  the desired count is within the acceptable range
Events:
pinaster00@pinaster00:~$

```

Figura 5.5: Pods autoscale con limits range impostati

```

rpinaster00@rpinaster00: ~ × rpinaster01@rpinaster01: ~ × rpinaster02@rpinaster02: ~ × rpinode00@rpinode00: ~ × dhexby@dhexby-Yoga: ~/Desktop ×
Every 5.0s: kubectl get hpa,pods -n demo rpinaster01: Fri Jun 5 11:17:24
2026
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
horizontalpodautoscaler.autoscaling/api Deployment/api cpu: 94%/60% 3 15 15 25h

NAME READY STATUS RESTARTS AGE
pod/api-5c67599cb4-4gl57 1/1 Running 0 93s
pod/api-5c67599cb4-cqcp8 1/1 Running 0 63s
pod/api-5c67599cb4-dv962 1/1 Running 0 48s
pod/api-5c67599cb4-fbgk6 1/1 Running 0 63s
pod/api-5c67599cb4-fnxr7 1/1 Running 0 48s
pod/api-5c67599cb4-g966x 1/1 Running 0 78s
pod/api-5c67599cb4-nc9h8 1/1 Running 0 93s
pod/api-5c67599cb4-pcr8b 1/1 Running 0 25h
pod/api-5c67599cb4-pwvjw 1/1 Running 0 26h
pod/api-5c67599cb4-rf5bh 1/1 Running 0 63s
pod/api-5c67599cb4-rl10p 1/1 Running 0 78s
pod/api-5c67599cb4-szl18 1/1 Running 0 63s
pod/api-5c67599cb4-xp5b6 1/1 Running 0 63s
pod/api-5c67599cb4-xq9t6 1/1 Running 0 93s
pod/api-5c67599cb4-z15c4 1/1 Running 0 25h
pod/api-5c67599cb4-z15c4 1/1 Running 0 25h
pod/frontend-bb97f5658-dzdf7 1/1 Running 0 26h
pod/frontend-bb97f5658-wgt72 1/1 Running 0 26h
pod/frontend-bb97f5658-z7lvb 1/1 Running 0 26h
pod/frontend-bb97f5658-zqntx 1/1 Running 0 26h
pod/redis-0 1/1 Running 0 26h

```

Figura 5.6: Check autoscale pods e relativo status

```

rpinaster00@rpinaster00:~$ kubectl top nodes
NAME              CPU(cores)   CPU%    MEMORY(bytes)  MEMORY%
rpinaster00       892m         22%     1653Mi         43%
rpinaster01       676m         16%     1420Mi         37%
rpinaster02       422m         10%     1424Mi         37%
rpinode00         438m        10%     927Mi          24%
rpinode01        537m        13%     2068Mi         54%
rpinode02        344m         8%      1168Mi         30%
```

6. Monitoring con Prometheus e Grafana

Stack di Monitoring

Prometheus è un database di tipo time-series che raccoglie metriche tramite *scraping HTTP* interrogando periodicamente gli endpoint `/metrics` dei pod, dei nodi e dei componenti k3s.

Su ogni nodo, **Node Exporter** gira ed espone metriche hardware (CPU, RAM, disk, rete).

kube-state-metrics espone lo stato degli oggetti Kubernetes.

Grafana si connette a Prometheus come datasource e permette di creare dashboard interattive.

6.1 Installazione kube-prometheus-stack via Helm

Listing 6.1: Da rpimaster00

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring

helm install prometheus-stack \
  prometheus-community/kube-prometheus-stack \
  - namespace monitoring \
  - set prometheus.prometheusSpec.retention=7d \
  - set grafana.adminPassword=pascal123 \
  - set grafana.service.type=NodePort \
  - set grafana.service.nodePort=30300 \
  - set prometheus.service.type=NodePort \
  - set prometheus.service.nodePort=30090 \
  - set alertmanager.service.type=NodePort \
  - set alertmanager.service.nodePort=30093 \
  - set nodeExporter.enabled=true \
  - set kubeStateMetrics.enabled=true

# attendiamo qualche secondo che tutti i pod siano 'Ready'
kubectl get pods -n monitoring - watch
```

Tabella 6.1: Endpoint del monitoring stack

Servizio	URL	Credenziali
Grafana	http://10.39.100.250:30300	admin / pascal123
Prometheus	http://10.39.100.250:30090	—
AlertManager	http://10.39.100.250:30093	—

6.2 Dashboard Grafana

Tabella 6.2: Dashboard Grafana preconfigurate solo da importare

Dashboard	ID	Contenuto
Kubernetes Cluster Overview	7249	CPU, RAM, pod count
Node Exporter Full	1860	Metriche hardware RPi
Kubernetes / Pods	6417	CPU/RAM per pod, restart
Kubernetes Deployments	8588	Repliche, rollout
etcd	3070	Leader election, latenza

Per importare: Grafana → **Dashboards** → **Import** → inserire l'ID → selezionare datasource Prometheus.

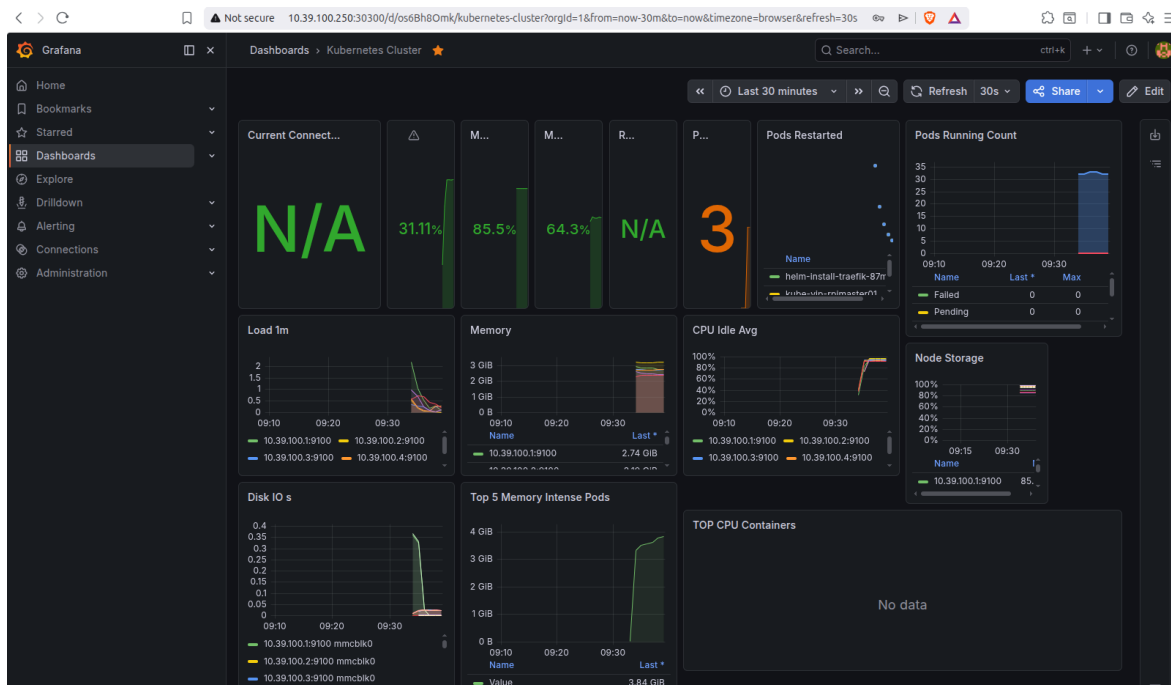


Figura 6.1: Schermata con metriche disponibili su Grafana

Video degli esperimenti Per una dimostrazione pratica del Monitoring delle metriche con Grafana e Prometheus, è disponibile il seguente video:

- Metrics Grafana

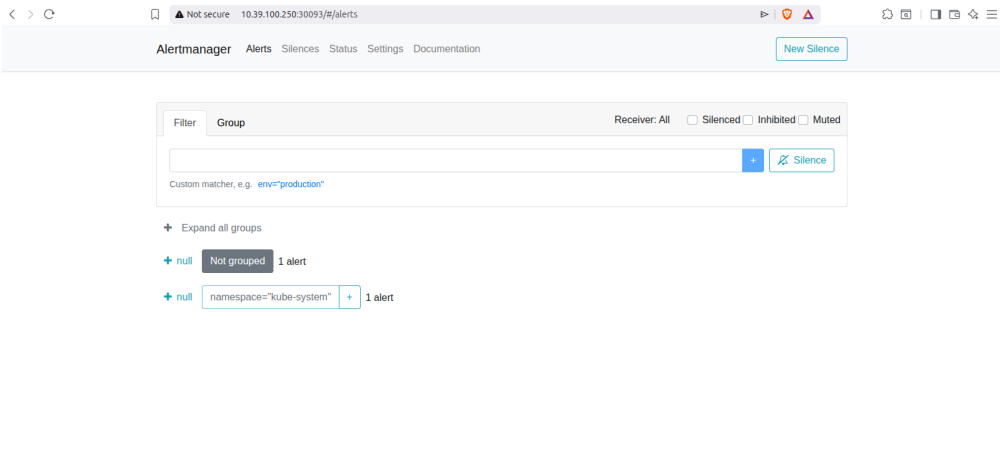


Figura 6.2: Pagina principale Alert Manager

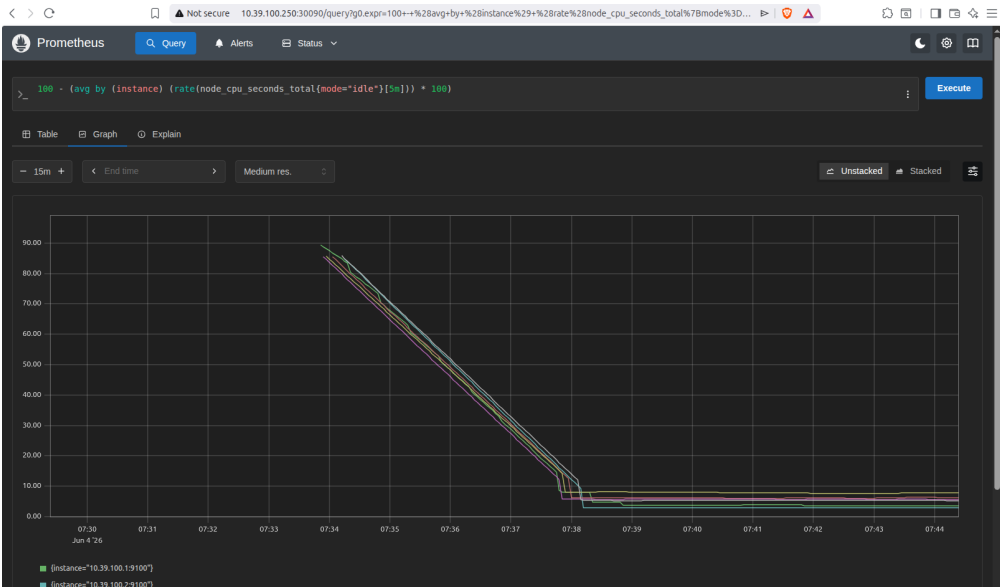


Figura 6.3: Esempio di query Prometheus

7. Load Testing e Test di Failover

7.1 Load Testing con k6

Utilizzo di **k6** per generare carico sull'applicazione demo e osservare l'HPA scalare le repliche:

Listing 7.1: loadtestHPADemo.js – Script k6

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 50 }, // ascesa a 50 utenti
    { duration: '5m', target: 200 }, // sale a 200 utenti
    { duration: '2m', target: 0 },   // descaling
  ],
  thresholds: {
    http_req_duration: ['p(90)<450'], // 90% richieste < 450
                                     ms
    http_req_failed: ['rate<0.1'],    // errori < 10%
  },
};

export default function() {
  const BASE = 'http://10.39.100.250';
  const headers = { Host: 'demo.cluster.local' };

  let res = http.get(`${BASE}/`, { headers });
  check(res, { 'frontend ok': r => r.status === 200 });

  res = http.get(`${BASE}/api/`, { headers });
  check(res, { 'api ok': r => r.status === 200 });

  sleep(0.5);
}
```

Listing 7.2: Esecuzione test (su pc host) e osservazione HPA

```
# installazione di k6 su PC host
sudo gpg -no-default-keyring \
  -keyring /usr/share/keyrings/k6-archive-keyring.gpg \
  -keyserver hkps://keyserver.ubuntu.com:80 \
  -recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.
gpg] \
https://dl.k6.io/deb stable main" \
| sudo tee /etc/apt/sources.list.d/k6.list
```



```

sudo apt update && sudo apt install k6

# lanciamo il test scritto precedentemente
k6 run loadtestHPADemo.js

# in un altro terminale, guardiamo lo scaling automatico
watch -n 5 kubectl get hpa, pods -n demo

```



```

j
dhexby@dhexby-Yoga:~/Desktop$ k6 run loadtestHPA.js

  execution: local
    script: loadtestHPA.js
    output: -

  scenarios: (100.00%) 1 scenario, 200 max VUs, 9m30s max duration (incl. graceful stop):
    * default: Up to 200 looping VUs for 9m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

  running (2m16.6s), 058/200 VUs, 7581 complete and 0 interrupted iterations
  default [=====>-----] 058/200 VUs  2m16.6s/9m00.0s

```

Figura 7.1: Esecuzione K6 load dal PC host

7.2 Test di Failover - High Availability

Come funziona il failover

Quando un master cade, **kube-vip** rileva la perdita del leader (grazie al mancato heartbeat ARP) e trasferisce il VIP a uno degli altri master in pochi secondi. I worker, connessi tramite VIP, non perdono la connessione all'API Server e i pod esistenti continuano a girare grazie al fatto di avere kubelet che è autonomo.

Quando un worker cade, il controller manager rileva il nodo *NotReady* dopo circa 30-40 secondi e reschedula i pod su altri nodi, in base al timer di eviction configurabile.

7.2.1 Fallimento di un Worker

Listing 7.3: Da rpimaster00

```

# attraverso "cordon" si impedisce nuovi scheduling su
  rpinode00
kubectl cordon rpinode00

# Drain: evict di tutti i pod da rpinode00
kubectl drain rpinode00 \
  - ignore-daemonsets \
  - delete-emptydir-data

# osserviamo i pod reschedularsi automaticamente
kubectl get pods -n demo -o wide - watch

```

```
# riabilitiamo il nodo dopo l'esperimento
kubectl uncordon rpinode00
```

Risultato: durante il drain del worker, i pod si reschedulano in ~30 secondi sugli altri worker disponibili. E' possibile monitorare il processo in tempo reale anche da Grafana setupato precedentemente attraverso le dashboard importate.

```
rpimaster00@rpimaster00:~$ kubectl cordon rpinode00
node/rpinode00 cordoned
rpimaster00@rpimaster00:~$ kubectl drain rpinode00 --ignore-daemonsets --delete-emptydir-data
node/rpinode00 already cordoned
Warning: ignoring DaemonSet-managed Pods: ingress-nginx/ingress-nginx-controller-jc9rs, monitoring/prometheus-stack-prometheus-node-exporter-w65g5
evicting pod monitoring/prometheus-stack-kube-prom-operator-78c5c76f74-6h5kz
evicting pod kube-system/helm-delete-traefik-tpocl
evicting pod demo/frontend-bb97f5658-zqntx
evicting pod demo/redis-0
pod/redis-0 evicted
pod/prometheus-stack-kube-prom-operator-78c5c76f74-6h5kz evicted
pod/frontend-bb97f5658-zqntx evicted
pod/helm-delete-traefik-tpocl evicted
node/rpinode00 drained
rpimaster00@rpimaster00:~$ kubectl get pods -n demo -o wide --watch
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
api-5c67599cb4-fnrx7	1/1	Running	0	36m	10.42.4.15	rpimaster01	<none>	<none>
api-5c67599cb4-pcr8b	1/1	Running	0	25h	10.42.1.13	rpimaster01	<none>	<none>
api-5c67599cb4-zl5c4	1/1	Running	0	25h	10.42.0.105	rpimaster00	<none>	<none>
frontend-bb97f5658-dzdfz	1/1	Running	0	26h	10.42.5.62	rpimaster02	<none>	<none>
frontend-bb97f5658-wcbjt	1/1	Running	0	34s	10.42.1.70	rpimaster01	<none>	<none>
frontend-bb97f5658-wgt72	1/1	Running	0	26h	10.42.4.3	rpimaster01	<none>	<none>
frontend-bb97f5658-z7lvb	1/1	Running	0	26h	10.42.2.32	rpimaster02	<none>	<none>
redis-0	0/1	Pending	0	33s	<none>	<none>	<none>	<none>

Figura 7.2: Cordon su rpinode00 e visualizzazione pods

7.3 Scenario: 1 Master Cade

Quorum Raft con 1 master down (2/3 disponibili)

Con 3 master etcd, il quorum richiede **2 nodi su 3**. Se cade 1 master, rimangono 2 nodi master attivi: il quorum è *mantenuto*.

Raft elegge un nuovo leader in pochi secondi, il cluster continua ad accettare scritture, schedulare nuovi pod, fare eventuali rollout e gestire lo scaling.

kube-vip trasferisce automaticamente il VIP al nuovo master leader. Dunque, possiamo notare come un singolo guasto sia completamente trasparente al carico di lavoro.

Listing 7.4: su rpimaster00 - failover

```
# in un terminale separato andiamo a monitorare il cluster
watch -n 1 "kubectl get nodes && echo ' - - ' && date"

# mentre su rpimaster00, fermiamo k3s, simulando un crash,
# spegnimento, problema)
sudo systemctl stop k3s

# monitoriamo la migrazione del VIP su rpimaster01 o
# rpimaster02 su un altro terminale
watch -n 1 "ping -c 1 10.39.100.250 && \
echo 'VIP OK' || echo 'VIP UNREACHABLE'"
```

Listing 7.5: Da rpimaster01 o rpimaster02 - verifichiamo l'operativita'

```
# verifichiamo che i nodi rimanenti siano in stato di 'Ready'
kubectl get nodes

# controlliamo che etcd abbia ancora quorum (2/3 nodi attivi)
sudo ETCDCTL_API=3 etcdctl \
  - endpoints=https://127.0.0.1:2379 \
  - cacert=/var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  - cert=/var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  - key=/var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  endpoint status - cluster -w table

# confermiamo che il cluster accetta ancora SCRITTURE
kubectl scale deployment api -n demo --replicas=4
kubectl get pods -n demo -o wide
```

Risultato atteso con 1 master down: failover VIP in 3-10 secondi. Il cluster rimane completamente operativo, potendo quindi avere nuovi pod schedulabili, eseguire possibili rollout e scritture etcd.

L'applicazione risponde con una brevissima interruzione durante il trasferimento VIP. I worker non richiedono alcun intervento manuale, astraendo e separando dunque il problema.

7.4 Scenario: 2 Master in failover

Perdita del Quorum Raft (1/3 disponibile)

Con solo 1 master su 3 rimasto attivo, il quorum Raft ($\geq 2/3$) viene **perso**. etcd entra in stato *read-only*, accettando dunque solo letture ma rifiuta qualsiasi tipo di scrittura. Il control plane non può più eseguire operazioni come schedulare nuovi pod, aggiornare deployment e/o creare risorse.

Tuttavia, i **pod già in esecuzione continuano a girare** sui worker: kubelet è autonomo e non dipende dall'API Server per mantenere i container attivi. Il ripristino richiede di riportare online almeno un secondo master.

Listing 7.6: Simulazione crash di 2 master su 3

```
# monitoriamo dal terzo master in un terminale separato
watch -n 2 "kubectl get nodes 2>&1 | head -20 && echo ' - - ' && date"

# su rpimaster00 fermiamo k3s
sudo systemctl stop k3s

# attendiamo 15-20 secondi, poi fermiamo anche rpimaster01
sudo systemctl stop k3s # su rpimaster01
```

Listing 7.7: Su rpimaster02 - osservazione con quorum perso

```
# Lettura: funziona ancora
kubectl get pods -A
```

```
kubectl get nodes -o wide

# Scrittura: deve fallire con errore etcd
kubectl scale deployment api -n demo --replicas=5

# I pod ESISTENTI continuano a girare sui worker:
ssh rpinode00@rpinode00.local "sudo crictl ps"
```

Tabella 7.1: Impatto per numero di master down

Master down	Quorum	Comportamento
0	3/3	Cluster completamente operativo
1	2/3	Completamente operativo. VIP failover automatico. Nessun impatto
2	1/3	Cluster in sola lettura. Pod esistenti continuano. Nuovi scheduling bloccati
3	0/3	Cluster non gestibile. Pod sui worker sopravvivono fino al prossimo riavvio

Risultato con 2 master down: etcd perde il quorum e il cluster entra in sola lettura. I pod esistenti sui worker *continuano a girare*. Il traffico applicativo già instradato da Nginx prosegue normalmente.

Video degli esperimenti Per una dimostrazione pratica del funzionamento e dei vari scenari sopra descritti riguardanti l'HA, è disponibile il seguente video:

- k3s cluster rpi - HA

8. Simulazione Split-Brain via Network Partition

Attraverso l'utilizzo di `iptables` è possibile isolare i master tra loro senza spegnerli, simulando una **partizione di rete**. I nodi sono “vivi” ma non si vedono reciprocamente (situazione realmente accaduta in numerosi sistemi distribuiti a causa di guasti agli switch, errori di configurazione del routing, problemi ai firewall o interruzioni parziali della connettività di rete.)

`etcd` entra in stallo ma i master credono ancora di essere leader.

A differenza dello spegnimento fisico, nella network partition, il nodo isolato rimane attivo e continua ad accettare richieste dal suo lato della partizione, generando potenzialmente risposte incoerenti se i client si connettono direttamente al suo IP fisico anziché al VIP.

8.1 Topologia del cluster prima del test

Listing 8.1: Inventario IP per il test split-brain

<code>rpimaster00</code>	<code>10.39.100.6</code>	<code><- master attuale, VIP</code>
<code>rpimaster01</code>	<code>10.39.100.3</code>	<code><- secondo master</code>
<code>rpimaster02</code>	<code>10.39.100.4</code>	<code><- terzo master</code>
<code>rpinode00</code>	<code>10.39.100.2</code>	<code><- worker</code>
<code>rpinode01</code>	<code>10.39.100.7</code>	<code><- worker</code>
<code>rpinode02</code>	<code>10.39.100.12</code>	<code><- worker</code>
<code>k3s-vip</code>	<code>10.39.100.250</code>	<code><- VIP flottante kube-vip</code>

Il nostro piano: si taglia la comunicazione tra `rpimaster00` e gli altri due master con `iptables`. I master continuano a girare, i worker continuano a girare, ma `rpimaster00` e `rpimaster01+02` non si vedono più. Ora dunque Raft dovrà decidere chi sarà il master VIP.

8.2 Preparazione del monitoraggio

Listing 8.2: Script `etcd-watch.sh` (da copiare su tutti i master)

```
cat > /tmp/etcd-watch.sh << 'EOF'
#!/bin/bash
sudo etcdctl \
  - endpoints=https://127.0.0.1:2379 \
  - cacert=/var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  - cert=/var/lib/rancher/k3s/server/tls/etcd/server-client.crt \
  - key=/var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  endpoint status -cluster -w table
EOF
chmod +x /tmp/etcd-watch.sh
```

```
scp /tmp/etcd-watch.sh pi@10.39.100.3:/tmp/
scp /tmp/etcd-watch.sh pi@10.39.100.4:/tmp/

watch -n 2 /tmp/etcd-watch.sh
```

8.3 Fase 1 - Fotografia dello stato iniziale

Listing 8.3: Su rpimaster00 – Terminale 1

```
sudo ETCDCCTL_API=3 etcdctl \
  - endpoints=https://127.0.0.1:2379 \
  - cacert=/var/lib/rancher/k3s/server/tls/etcd/server-ca.crt \
  - cert=/var/lib/rancher/k3s/server/tls/etcd/server-client. \
    crt \
  - key=/var/lib/rancher/k3s/server/tls/etcd/server-client.key \
  endpoint status - cluster -w table
```

Tabella 8.1: Output dell'endpoint status prima della partizione di rete

Endpoint	ID	Version	Leader
10.39.100.6:2379	8e9e05c52164694d	3.5.x	Yes
10.39.100.3:2379	91bc3c398fb3c146	3.5.x	No
10.39.100.4:2379	fd422379fda50e48	3.5.x	No

Listing 8.4: Su rpinode00 - Terminale 4

```
# Loop che pinga il VIP ogni 0.5 secondi
while true; do
  STATUS=$(curl -sk \
    - max-time 1 \
    https://10.39.100.250:6443/healthz 2>&1)
  echo "$(date '+%H:%M:%S.%3N') | $STATUS"
  sleep 0.5
done
```

8.4 Fase 2 - Avvio del monitoring continuo

Listing 8.5: Su rpimaster01 - Terminale 2

```
watch -n 0.5 "  
  echo '=== NODES ===' && \  
  sudo kubectl get nodes -no-headers 2>&1 && \  
  echo '' && \  
  echo '=== VIP location ===' && \  
  ip addr show eth0 | grep 10.39.100.250 || echo 'VIP non e'  
  qui '  
"
```

Listing 8.6: Su rpimaster02 - Terminale 3

```
watch -n 0.5 "  
  echo '=== NODES ===' && \  
  sudo kubectl get nodes -no-headers 2>&1 && \  
  echo '' && \  
  echo '=== VIP su questo nodo? ===' && \  
  ip addr show eth0 | grep 10.39.100.250 || echo 'no VIP qui '  
"
```

8.5 Fase 3 - creazione della partizione di rete

Configurando iptables, come nel listing successivo, su **rpimaster00** si bloccano tutti i pacchetti IP verso e dai due master peer. Il nodo continua ad esistere fisicamente come anche la rete fisica è intatta, ma a livello IP non vede più nessuno dei suoi pari.

Listing 8.7: Su rpimaster00 - ESEGUIRE IN RAPIDA SEQUENZA

```
# Blocca traffico IN e OUT verso rpimaster01  
sudo iptables -A INPUT -s 10.39.100.3 -j DROP  
sudo iptables -A OUTPUT -d 10.39.100.3 -j DROP  
  
# Blocca traffico IN e OUT verso rpimaster02  
sudo iptables -A INPUT -s 10.39.100.4 -j DROP  
sudo iptables -A OUTPUT -d 10.39.100.4 -j DROP  
  
echo "PARTIZIONE ATTIVA - $(date)"
```

Da questo momento **rpimaster00** è isolato. Non sa che gli altri due stanno ancora parlando tra loro.

8.6 Fase 4 - Osservazione

Listing 8.8: Su rpimaster00 - monitoring etcd locale

```
watch -n 2 "
  sudo ETCDCTL_API=3 etcdctl \
    - endpoints=https://127.0.0.1:2379 \
    - cacert=/var/lib/rancher/k3s/server/tls/etcd/server-ca.
      crt \
    - cert=/var/lib/rancher/k3s/server/tls/etcd/server-client.
      crt \
    - key=/var/lib/rancher/k3s/server/tls/etcd/server-client.
      key \
    endpoint status -w table 2>&1
"
```

8.6.1 Timeline degli eventi osservati

Tabella 8.2: Sequenza temporale eventi durante la partizione

Tempo	Su rpimaster00	Su rpimaster01/02
T+0s	IS LEADER = true (non sa niente)	Heartbeat da master00 assente
T+10s	IS LEADER = true (timeout non scattato)	Contatore mancati heartbeat aumenta
T+25s	Errore: <code>request timed out</code>	Uno tra master01/02 eletto nuovo leader
T+30s	Richieste <code>kubectl</code> in timeout	VIP migra al nuovo leader
T+30s	—	<code>ip addr</code> mostra 10.39.100.250 sul nuovo master

I tempi riportati nella tabella precedente sono una stima abbastanza accurata. Potrebbero essere necessari pochi secondi in più o in meno talvolta.

8.7 Fase 5 - interrogazione durante la partizione

Listing 8.9: Da rpimaster01 (isola con quorum 2/3)

```
# proviamo a vedere se il cluster accetta ancora scritture
  creando un nuovo namespace
sudo kubectl create namespace split-brain-test

# proviamo a schedulare un nuovo pod
sudo kubectl run canary \
  - image=busybox \
  - restart=Never \
  -n split-brain-test \
  - sleep 300

sudo kubectl get pods -n split-brain-test -o wide
```

Listing 8.10: Da rpimaster00 (nodo isolato, senza quorum)

```
# rpimaster00 non ha il quorum
sudo kubectl get nodes 2>&1
# Risposta attesa      un classico 'timeout '

sudo kubectl create namespace split-brain-test-2 2>&1
# Risposta attesa sempre 'timeout '
```

8.8 Fase 6 - Rimozione della partizione

Listing 8.11: Su rpimaster00 - rimozione delle regole iptables impostate in fase 3

```
# Rimuovi le regole nell'ordine inverso
sudo iptables -D OUTPUT -d 10.39.100.4 -j DROP
sudo iptables -D INPUT  -s 10.39.100.4 -j DROP
sudo iptables -D OUTPUT -d 10.39.100.3 -j DROP
sudo iptables -D INPUT  -s 10.39.100.3 -j DROP

echo "PARTIZIONE RIMOSSA - $(date)"

# Verifica che le regole siano sparite
sudo iptables -L INPUT  -n | grep DROP
sudo iptables -L OUTPUT -n | grep DROP
```


8.10 Recupero da situazioni anomale

Dopo la **partizione di rete** creata precedentemente o un blocco imprevisto di uno dei nodi rpimaster, è necessario ripristinare la comunicazione tra i nodi e garantire che tutti i componenti del cluster tornino a uno stato coerente.

La procedura illustrata di seguito mostra alcuni passaggi, solo i principali, per un eventuale recupero disperato: riavvio dei servizi principali, rimozione temporanea delle regole firewall applicate durante il test e verifica dello stato del cluster. Questi comandi permettono di riportare i nodi master (nel nostro caso solo rpimaster00) in condizione Ready e di accertarsi che i pod vengano schedulati correttamente, evitando situazioni di blocco prolungato.

Listing 8.13: Comandi di emergenza post-test

```
# se rpimaster00 rimane bloccato per qualche motivo, possiamo
  forzare il restart dela daemon k3s
sudo systemctl restart k3s

# flush completo delle regole iptables (ATTENZIONE: rimuove
  qualsiasi regola imposta, dunque usare con precauzione se
  sono state applicate altre regole per altri scopi)
sudo iptables -F INPUT
sudo iptables -F OUTPUT

# verifica dello stato attuale del cluster
sudo kubectl get nodes
sudo kubectl get pods -A | grep -v Running
```

9. Conclusioni

L'infrastruttura realizzata rappresenta un ambiente che riproduce, o cerca di riprodurre in parte, i comportamenti di un cluster Kubernetes di produzione, con la comodità però di essere completamente gestibile e ripristinabile su hardware dedicato. Grazie al progetto siamo riusciti a dimostrare la robustezza di un'architettura "economica" rispetto agli standard di produzione attraverso l'utilizzo di **k3s** in **alta disponibilità** su hardware ARM grazie a Raspberry pi 400.

I risultati principali ottenuti:

- **High Availability attraverso kube-VIP:** la perdita di 1 master su 3 è completamente trasparente al carico di lavoro, con failover del VIP in 3-10 secondi.
- **Comportamento corretto in read-only:** con 2 master down, il cluster entra correttamente in sola lettura e i pod esistenti continuano a girare senza interruzioni.
- **Monitoring integrato:** grazie alla configurazione dello stack composto da **Prometheus/Grafana**, esso ci fornisce visibilità completa su tutte le fasi dei test/esperimenti eseguiti, rendendo di fatto ogni evento misurabile con precisione.
- **Scalabilità orizzontale sui nodi:** l'**HPA** ha dimostrato di rispondere correttamente al carico generato da k6, scalando le repliche da 3 a 15 in poco tempo.
- **Split-brain gestito da Raft:** il protocollo **Raft** garantisce che solo la partizione con quorum maggioritario continui a operare, prevenendo dati e situazione inconsistenti, soprattutto nell'eventuale ricongiungimento dei master.

Video degli esperimenti Per una dimostrazione pratica di tutti gli esperimenti riportati, è disponibile la seguente cartella:

- [Link Folder Esperimenti registrati](#)

A. Riferimenti e Risorse

- Documentazione ufficiale k3s: <https://docs.k3s.io>
- kube-vip: <https://kube-vip.io>
- Flannel CNI: <https://github.com/flannel-io/flannel>
- kube-prometheus-stack: <https://github.com/prometheus-community/helm-charts>
- etcd Raft: <https://github.com/etcd-io/raft>
- Building Distributed Key Value Database using etcd Raft: <https://medium.com/@mohiteakshay2020/building-distributed-key-value-database-using-raft-part-i-d1d5e38fee82>
- k6 load testing: <https://k6.io/docs/>
- Nginx Ingress Controller: <https://kubernetes.github.io/ingress-nginx/>